Classi e oggetti in UML

Classi

In UML una **classe** è un modello astratto che descrive un insieme di oggetti.

Definisce:

- **Attributi** (proprietà, campi con tipo)
- **Metodi** (operazioni, comportamenti — non elencati qui ma possibili)

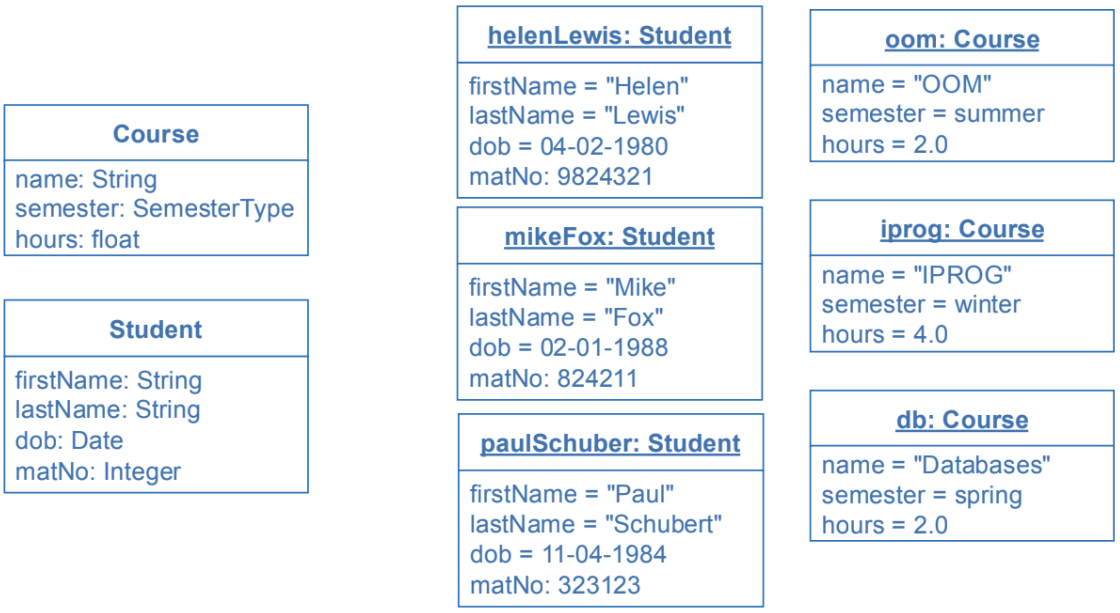


Diagramma delle classi UML

Cos'è

Il **diagramma delle classi** è un diagramma statico che descrive:

- Le **classi** (tipi di oggetti) di un sistema.

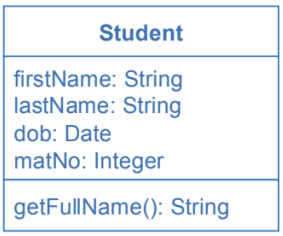
- Le **relazioni permanenti** tra queste classi.
- Le **proprietà** e le **operazioni** delle classi.

È una rappresentazione strutturale: mostra cosa compone il sistema, non il suo comportamento dinamico.

Struttura di una classe UML

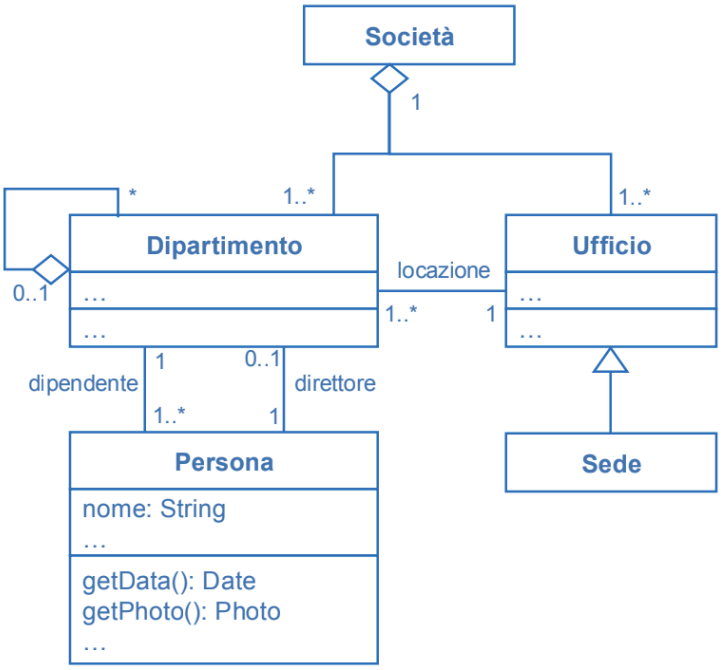
Una classe si rappresenta con un rettangolo diviso in tre sezioni:

1. Nome della classe (in alto).
2. Attributi (con il tipo).
3. Operazioni/metodi.



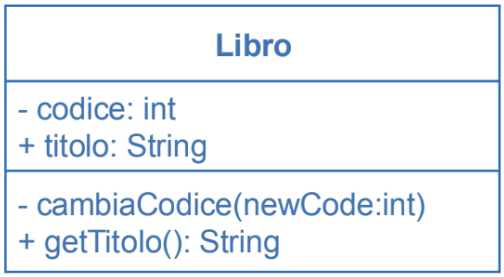
Esempio: Concetti individuati

- **Società** (classe principale, composta da dipartimenti e uffici).
- **Dipartimento** (ha direttore, dipendenti, appartiene a un ufficio, può avere dipartimenti subordinati).
- **Ufficio** (sede fisica, contiene dipartimenti).
- **Dipendente** (lavoratore del dipartimento, uno può essere direttore).



Struttura grafica di una classe in UML

Una classe è rappresentata da un rettangolo suddiviso in tre comparti:



1. **Nome della classe** (obbligatorio)
   - Sempre al **singolare**
   - In **maiuscolo** (es. `LIBRO`)
2. **Attributi** (facoltativo)
3. **Operazioni / Metodi** (facoltativo)

Livello di astrazione nel diagramma delle classi

Quando	Cosa mostrare	Cosa evitare
Descrizione del dominio (concettuale)	- Solo attributi che caratterizzano l'elemento del dominio (es. `titolo`, `autore` in un libro)- Relazioni con altre classi	- Dettagli implementativi (es. `ID` come chiave primaria)- Operazioni banali (setter/getter)- Modificatori di visibilità (+, –, #)

Quando	Cosa mostrare	Cosa evitare
Descrizione architetturale / implementativa (design del software)	- Attributi con tipi precisi (es. `codice: String`)- Operazioni/metodi con parametri e tipo di ritorno- Modificatori di visibilità (+ pubblico, – privato, # protetto)- Associazioni con molteplicità	- Concetti troppo astratti non legati al codice
Descrizione dettagliata del codice (blueprint)	- Tutti gli attributi e i metodi realmente implementati- Specifica dei tipi- Modificatori di visibilità- Dettagli come costruttori, interfacce implementate, ecc.	- Informazioni non presenti nel codice eseguibile

Sintassi di un attributo UML

```
[visibilità] nome: tipo [molteplicità] = valoreIniziale {proprietà}
```

- **visibilità**:
  - `+` pubblico
  - `–` privato
  - `#` protetto
  - `~` package (visibile solo all'interno del package)
- **nome**: identificatore dell'attributo (es. `colore`, `nome`, `secondoNome`).
- **tipo**: tipo di dato (es. `String`, `Integer`, `Boolean`, o una classe).
- **molteplicità** *(opzionale)*: indica il numero di valori che l'attributo può assumere.
  - `[1]` (default) → un solo valore obbligatorio.
  - `[0..1]` → opzionale (può essere null).
  - `[0..*]` o `[*]` → insieme arbitrario di valori.

- `[3]` → esattamente tre valori.

- **valoreIniziale** *(opzionale)*: valore di default dell'attributo.
- **proprietà** *(opzionali, tra `{}`)*: vincoli sul dominio dei valori.
  - `{ordered}` → l'insieme di valori ha un ordine.
  - `{unique}` → i valori devono essere distinti.
  - `{>0, <10}` → vincoli sui valori numerici (intervallo).

Esempi commentati

Attributo UML	Significato
`nome: String`	Ogni oggetto ha obbligatoriamente un nome (stringa).
`secondoNome: String [0..1]`	L'attributo può essere assente (null) oppure avere un solo valore.
`colore: Integer [3] {ordered}`	Tre interi ordinati (es. RGB).
`voti: Integer [0..*] {unique}`	Insieme (senza duplicati) di voti interi.
`eta: Integer = 18 {>0, <120}`	L'età parte da 18, deve essere maggiore di 0 e minore di 120.

Esempi di Attributi UML

Attributo UML	Significato
`n: char`	Attributo di tipo carattere, tipo base.
`n: String`	Attributo stringa, tipo base predefinito.
`g: Gra`	Attributo che fa riferimento a una classe `Gra` definita nel modello.
`n: Integer = 1 {>= 0}`	Intero non negativo, valore iniziale 1.
`p: Integer [2] {>0, ordered}`	Array di 2 interi positivi, ordinati → rappresenta un punto (x,y) nel primo quadrante.
`nome: String [1..2] {ordered, unique}`	Ogni istanza deve avere almeno un nome, può avere al massimo due, distinti e ordinati

Attributo UML	Significato
	(es. “Mario Luigi”).

Modificatori di visibilità in UML

Simbolo	Nome	Significato
`+`	**public**	Attributo/operazione accessibile ovunque la classe sia visibile.
`#`	**protected**	Accessibile solo nella classe e nelle sue sottoclassi.
`-`	**private**	Accessibile solo all’interno della classe stessa.
`~`	**package**	Accessibile solo da classi nello stesso package.

Attributi (Sintassi UML)

```
visibilità nome: tipo [molteplicità] = valoreDefault {vincoli}
```

Esempi

- `n: char` → attributo di tipo carattere.
- `n: String` → attributo di tipo stringa.
- `g: Gra` → attributo di tipo personalizzato (classe `Gra`).
- `n: Integer = 1 {>= 0}` → intero non negativo, inizialmente `1`.
- `p: Integer [2] {>0, ordered}` → array di 2 interi positivi ordinati (es: un punto del piano).
- `nome: String [1..2] {ordered, unique}` → da 1 a 2 nomi, ordinati e diversi tra loro.

Visibilità

- `+` → **public** (visibile ovunque)
- `#` → **protected** (visibile alle sottoclassi)
- → **private** (visibile solo nella classe)
- `~` → **package** (visibile solo nel package)

Operazioni (Metodi)

```
visibilità nome(listaParametri): tipoRitorno
```

listaParametri → può essere vuota oppure composta da dichiarazioni:

```
nomeParametro: tipo = valoreDefault
```

tipoRitorno → opzionale (se non specificato = void).

Regola generale

- **Attributo**: usa un tipo semplice o già definito, quando il concetto non ha bisogno di dettagli propri.
- **Classe**: crea una nuova classe quando il concetto ha proprietà, comportamenti o relazioni significative proprie.

Esempio: Un libro ha uno o più autori

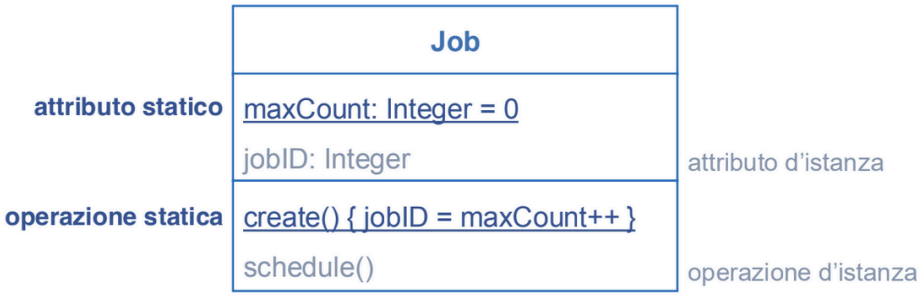
- autore come attributo di Libro: possiamo specificare solo il nome



- autore come classe: può avere attributi propri e eventualmente operazioni



Campi statici



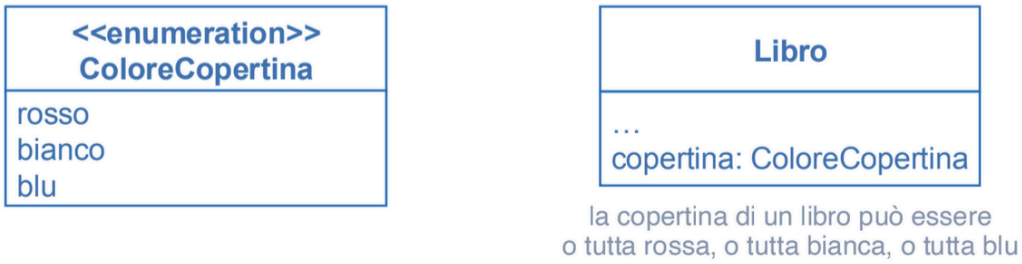
Enumerazioni

Le enumerazioni sono usate per specificare un insieme di valori prefissati

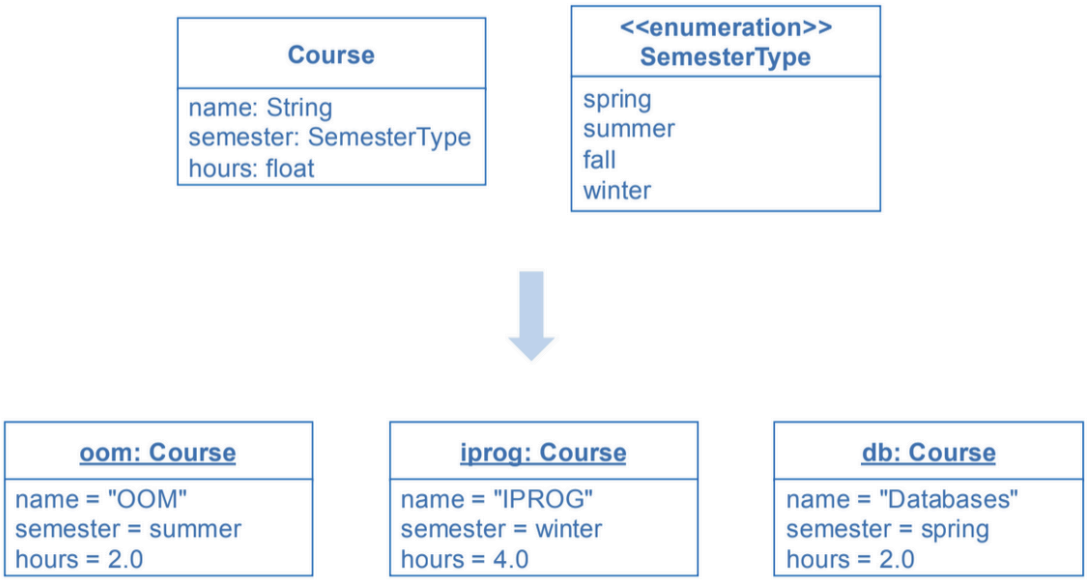
- lista completa di tutti i valori che gli attributi di un determinato tipo possono assumere

In UML sono rappresentate da classi

- etichettate dallo stereotipo <<enumeration>>
  • con un nome (il tipo) e l'insieme di valori che gli attributi di quel tipo possono assumere



- **esempio:**



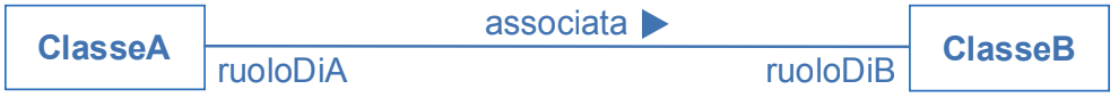
RELAZIONI IN UML

Una **relazione** rappresenta un **legame tra due o più oggetti**, tipicamente istanze di **classi diverse**.

Relazioni tra classificatori	Relazioni tra oggetti
Associazione Aggregazione Composizione	Collegamento Aggregazione Composizione
Generalizzazione	(non definita)
Realizzazione	(non definita)
Dipendenza (d'uso, d'istanza, ecc.)	

ASSOCIAZIONE

- Rappresentata con **una linea retta** che collega due classi.
- Descrive un **legame logico e semantico** tra istanze di quelle classi.



Elementi caratterizzanti

Elemento	Descrizione
Nome	Tipicamente un **verbo** che esprime la relazione (es. *scrive*, *contiene*, *gestisce*)
Ruoli	Sostantivi che indicano il ruolo di ciascuna classe nella relazione (es. *autore*, *libro*)
Verso di lettura	Opzionale; indica da che lato leggere l'associazione
Molteplicità	Specifica il numero di oggetti coinvolti nella relazione (es. 1, 0..*, 1..*)

ASSOCIAZIONI O ATTRIBUTI?

Nel **modello concettuale** si usano **associazioni**.

Nel **modello logico o nel codice**, queste vengono tradotte in **attributi**.

Esempio

Modello concettuale:

> Libro — associazione — Autore

Traduzione nel modello logico (codice):

```
Libro
autore: Autore [1..*]

Autore
nome: String
dataNascita: Date
```

VINCOLI DI MOLTEPLICITÀ

La **molteplicità** specifica **quanti oggetti di una classe** possono essere **associati** a un singolo oggetto dell'altra classe in una relazione (associazione).

Sintassi generale

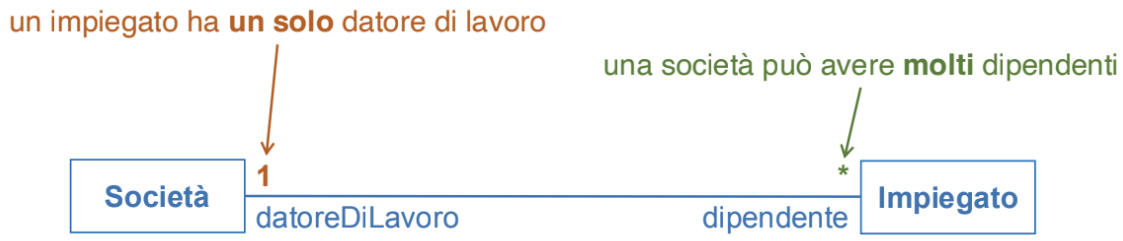
Elemento	Descrizione
`1`	Uno e un solo oggetto (default, può essere omesso)
`0..1`	Nessun oggetto oppure uno solo (opzionale)
`*`	Qualsiasi numero di oggetti (zero o più)
`1..*`	Almeno uno, ma numero non limitato
`n..m`	Da *n* a *m* oggetti (limiti inferiori e superiori espliciti)
`n..n`	Equivalente a `n`

Elemento	Descrizione
`0..*`	Equivalente a `*`

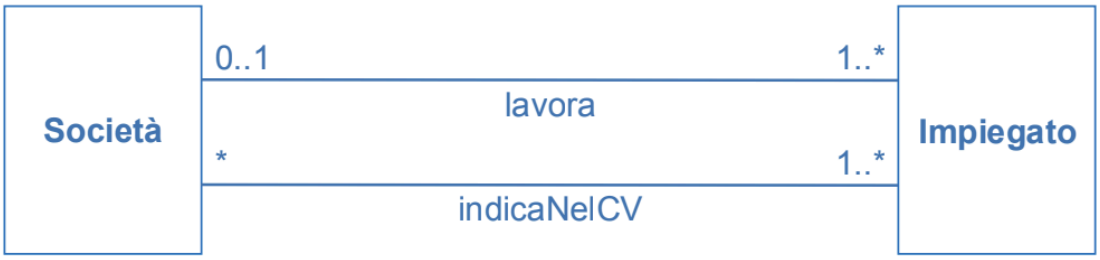
ESEMPIO PRATICO

Associazione	Molteplicità	Significato
Società — Impiegato	`1` — `*`	Una **società** può avere **molti impiegati**, ma ogni **impiegato** ha **una sola società** come datore di lavoro.
Veicolo — Ruota	`1` — `2..4`	Ogni **veicolo** ha **da 2 a 4 ruote**, e ogni **ruota** appartiene a **un solo veicolo**.
Utente — Account	`1` — `0..1`	Un **utente** può avere **al massimo un account**, e un **account** appartiene a **un solo utente**.

Schema UML d'esempio



Ci possono essere anche più associazioni tra due classi



Esempio tipico:

Classe `Persona`

- Un oggetto `Persona` può essere il *genitore* di un'altra `Persona`
- Allo stesso tempo, può essere anche il *figlio* di un'altra `Persona`

In UML si rappresenta con una **freccia che parte e torna sulla stessa classe**, e **i ruoli** servono a chiarire la natura dei due lati della relazione.

Associazioni Riflessive

Un'associazione riflessiva mette in relazione un'entità con se stessa



Aggregazione

Composizione

Una stampante può essere collegata (nel tempo) a computer diversi. Una stampante esiste anche senza computer.

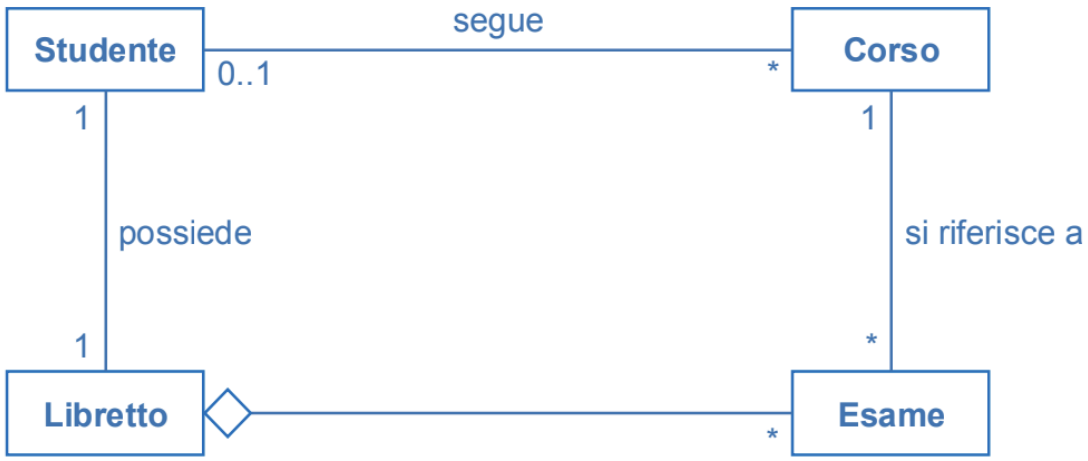
Se il computer viene distrutto, la stampante continua a esistere.

NB: l'aggregazione non ha un un nome

Un pulsante appartiene ad una sola stampante. Un pulsante non esiste senza la sua stampante.

Se la stampante viene distrutta, vengono distrutti anche i suoi pulsanti.

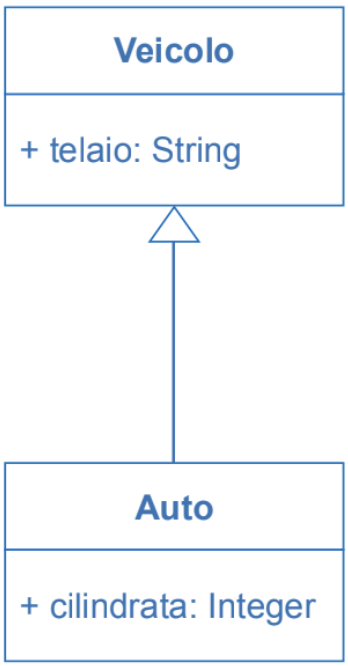
NB: La composizione non ha un nome



Generalizzazione

Relazione tra un elemento **generico (G)** e uno più **specializzato (S)**.

- **S** è consistente con **G**, ma contiene più informazione.
- **Principio di sostituzione di Liskov:** S può essere usato al posto di G.
- **Intuitivamente:** S è un tipo di G.



Generalizzazione ed Ereditarietà

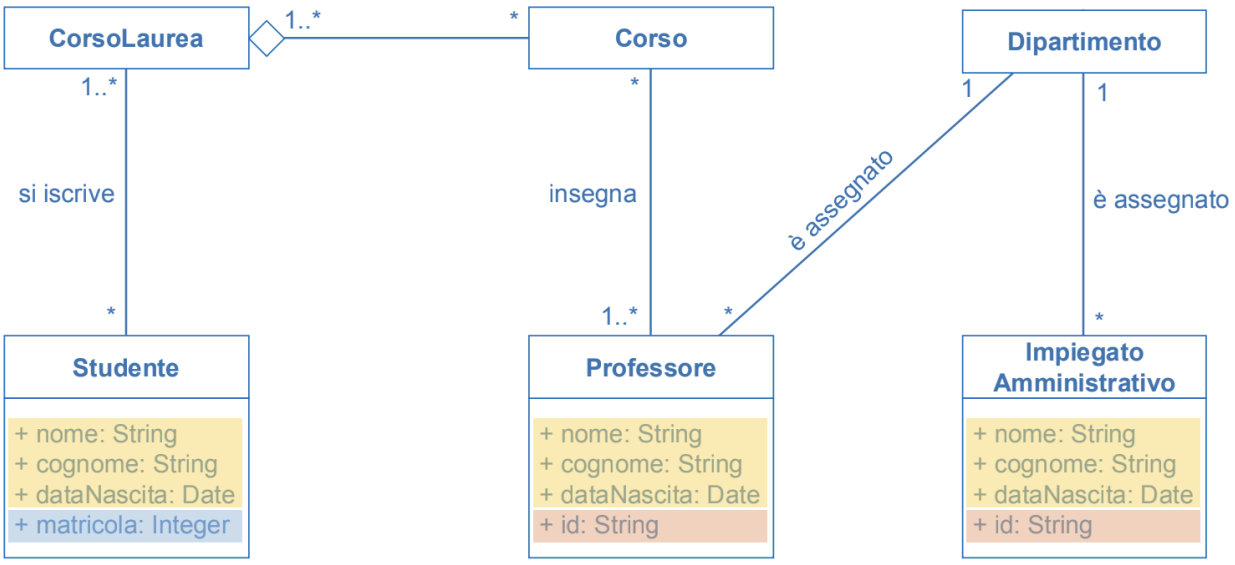
Una **superclasse** generalizza le sue **sottoclassi**.

Le sottoclassi **ereditano** caratteristiche dalla superclasse:

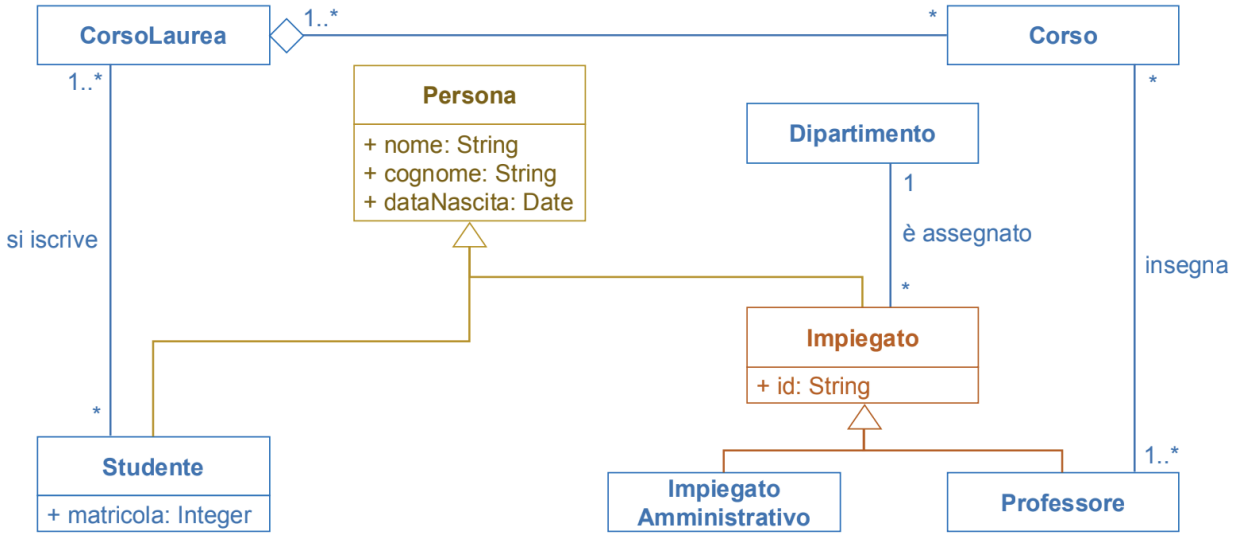
- attributi
- operazioni
- relazioni
- vincoli

Le sottoclassi possono inoltre:

- **aggiungere** nuove caratteristiche
- **ridefinire** le operazioni



Non è ottimale essendo che Studente, Professore e Impiegato amministrativo hanno campi in comune



Esempio ottimizzato

Dipendenze

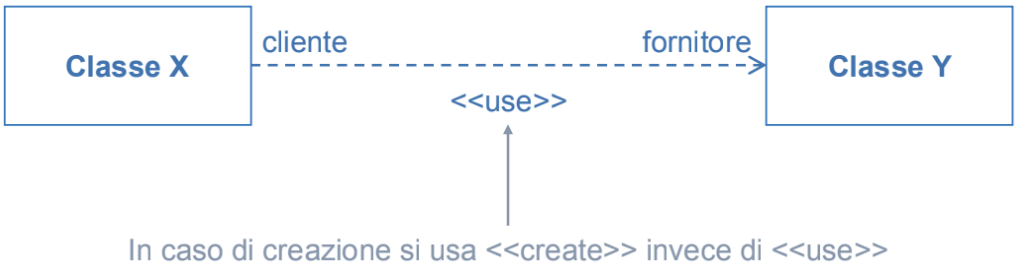
Una dipendenza è una relazione tra una classe cliente e una classe fornitore

- il cliente dipende dal fornitore
- una modifica nel fornitore può influenzare il cliente



Alcune dipendenze comuni

- Un parametro di un'operazione di X è di tipo Y
- Un'operazione di X restituisce un oggetto di tipo Y
- Un'operazione di X crea dinamicamente un oggetto di tipo Y



Individuare le Classi di Analisi (del Dominio)

FAQ: Cosa sono le classi di analisi?

Corrispondono a **concetti concreti del dominio**.

- Per esempio, ai concetti descritti nel **glossario**.
- Una **classe di analisi** viene spesso raffinata in **una o più classi di progettazione**.
- Evitare di introdurre **classi di progettazione** in questa fase.

FAQ: Quali tecniche usare per individuare le classi di analisi?

Non esiste un **metodo automatico** che le estragga da un documento in linguaggio naturale.

Tuttavia, è possibile utilizzare **alcune metodologie note** per supportare l'individuazione.

Classi di Analisi: Caratteristiche (e Tip)

- Astrazione di uno **specifico elemento del dominio**.
- Devono avere un **numero ridotto di responsabilità (funzionalità)**.
- Evitare di definire **classi "onnipotenti"**.

  > Attenzione quando si chiamano "sistema", "controllore", ecc.

- Evitare **funzioni travestite da classi**.
- Evitare **gerarchie di ereditarietà profonde (≥ 3)**.

Livello di astrazione/dettaglio

- Specificare **operazioni e attributi** solo quando **veramente utili** e **ad alto livello** (limitare tipi, valori, ecc.).
- **Non inventare nulla**, ma basarsi su:
  - quanto scritto nel **documento**
  - **confronto con clienti o utenti**

Classi di Analisi: Tecniche di Identificazione

Approccio *data-driven* (tipico della fase di analisi)

- Si identificano i **dati del sistema** (es. trovando i sostantivi).
- Si raggruppano in **classi**.

Approccio *responsibility-driven* (soprattutto durante la progettazione)

- Si identificano le **responsabilità**.
- Si suddividono in **classi**.

Passi dell'Analisi Nome-Verbo

1. **Individuazione delle classi**
2. **Assegnazione di attributi e responsabilità** alle classi
3. **Individuazione di relazioni** tra le classi

Problemi Ricorrenti

- **Tagliare le classi inutili**, ad esempio gestendo casi di **sinonimia**.
- **Individuare le classi nascoste**, ossia concetti impliciti del dominio del problema non menzionati esplicitamente.

Mappatura Sintattica

- **Sostantivi → classi o attributi**
- **Verbi → operazioni (responsabilità)**



Esempio

In un sistema di gestione degli orari delle lezioni di un corso universitario:

- La descrizione testuale potrebbe **non menzionare mai l'aula**,
- ma l'aula deve comunque essere **inserita nel modello** come classe.

Classi UML (IS) Entità (Basi di Dati)

- **Nome singolare** – rappresentano il “tipo” di un oggetto
- Per aggregare istanze: usare nomi come `ListaDiQualcosa`



- **Nome plurale** – rappresentano collezioni di oggetti
- Un’unica entità rappresenta **tutte le istanze** di un concetto (`Qualcosa`)



DIAGRAMMA DEGLI OGGETTI

Un oggetto si rappresenta con due sole sezioni

- nome dell’oggetto e della classe che istanzia, sottolineati
- lista degli attributi (opzionale e rarissima)
- nome attributo
  • tipo attributo (ridondante, meglio ometterlo)
  • valore attributo

